Turbofan Engine Remaining Useful Life Prediction

An LSTM Regressor on NASA C-MAPSS (FD001) | Exam 3 — Own ML Project

1. Problem Relevance

This project predicts Remaining Useful Life (RUL, in operating cycles) for aircraft turbofan engines from a rolling window of 15 sensor channels (temperatures, pressures, speeds, and flow measurements taken at the fan, compressor, and turbine stages). This is the aerospace instance of predictive maintenance: airlines and MRO (maintenance, repair, overhaul) providers schedule engine removal and part replacement around estimated remaining life rather than a fixed calendar interval, which is both a safety requirement (an engine must never fly past the point of unsafe degradation) and a cost problem (removing an engine too early wastes remaining useful life; removing it too late risks an in-flight fault). The asymmetry is explicit and quantified in the evaluation itself: the NASA/PHM08 scoring function used throughout Section 5 penalizes a late prediction — the model saying an engine has more life left than it actually does — far more heavily than an early, conservative one, mirroring exactly why maintenance schedulers err toward caution.

2. Data Choice

Source: NASA's C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) Turbofan Engine Degradation dataset, from the NASA Prognostics Center of Excellence (PCoE) data repository — a well-documented benchmark originally released for the 2008 PHM Data Challenge (Saxena et al., "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation", PHM08). This project uses subset FD001: 100 training engines and 100 test engines, a single operating condition, and a single fault mode (HPC — high-pressure compressor — degradation).

Each engine is a multivariate time series: it starts in a healthy state with unknown initial wear, degrades over time, and reaches failure at the end of its training trajectory (test trajectories are truncated at an arbitrary earlier point, with the true remaining life at that point given separately in RUL_FD001.txt). This sequential run-to-failure structure is the key reason this dataset was chosen over the previously-scoped single-snapshot automotive dataset (see project history / prior iteration): it is genuine time-series data, which finally makes a sequence model (LSTM) a legitimate, testable choice rather than a purely theoretical discussion point.

Data Quality Audit

Check	Result	Consequence
Missing values	None (26/26 raw columns)	No imputation needed
Train/test structure	100 train units run to failure; 100 test units truncated early, RUL given separately	Test RUL must be read from RUL_FD001.txt, never re-derived as max_cycle − t
Operating condition	op_setting 1–3 std ≈ 0 across all 20,631 training rows	Single condition confirmed — no per-regime normalization needed (unlike FD002/FD004); op settings dropped as zero-information
Constant sensors	6 of 21 sensors exactly constant (std < 1e-3): sensors 1, 5, 10, 16, 18, 19	Dropped — zero-information for this subset; 15 sensors retained
Trajectory length	Train: 128–362 cycles/unit. Test: 31–303 cycles/unit	Sets the sliding-window length (§3): must be ≤ 31 to avoid padding on real data
Train/test unit IDs	Both files number units 1–100, but these are independent, unrelated engines	Never joined across files by unit number (§3)
Simulation vs. real data	C-MAPSS is a physics-based simulation, not real fleet telemetry	Report figures are a methodology demonstration; see Limitations (§6)

Expected Limitations

- Simulated data: C-MAPSS is a physics-based simulation (validated against real engine behavior by NASA), not real fleet telemetry — a genuine production system would need to be revalidated on real sensor data before deployment.
- Single operating condition, single fault mode (FD001 only): FD002/FD004 add six operating regimes and a second fault mode, which is out of scope here — no claim is made about cross-condition generalization.
- 100 training engines is workable for a small LSTM but modest by deep-learning standards — supports the case for a small, heavily-regularized architecture (§3) rather than a large one.
- RUL beyond the 125-cycle cap is not distinguished by the model by construction (§3) — this is a deliberate modeling convention, not a data defect, but it is a real constraint on what the model can express.

3. Model Setup: Preprocessing & Architecture

RUL labeling: train RUL(t) = min(max_cycle(unit) − t, 125) — a standard piecewise-linear cap (e.g. Zheng et al., 2017). Early-life cycles carry no visible degradation signal, so forcing the model to regress an ever-growing uncapped target from near-identical healthy readings adds gradient noise without a learnable signal; capping concentrates model capacity on the informative near-failure region. The same cap is applied to the test ground truth from RUL_FD001.txt for the headline metric (unclipped test RMSE reported separately as 14.0 cycles, for transparency).

Windowing: sliding windows of length 30 cycles (stride 1 for training). 30 is the largest window that still leaves a 1-cycle margin under the shortest test trajectory (31 cycles), so every test engine yields one full window with no padding needed on this subset — and matches the value most commonly used in the C-MAPSS literature for FD001. Test-time evaluation uses each engine's final 30 recorded cycles, compared against its one RUL_FD001.txt value.

Leakage control: the 100 training engines are split 80/20 into train/val by unit number (before windowing), not by row — adjacent windows from the same engine overlap by up to 29 of 30 timesteps and are near-duplicates, so a row-level split would leak near-identical windows across train/val and produce deceptively optimistic validation error. StandardScaler is fit on the 80 train units' rows only, then applied to val and test.

```
Input(30 cycles x 15 sensors, standardized)
  -> LSTM(input=15, hidden=64, 1 layer, batch_first)
  -> final hidden state (64)
  -> Dropout(0.3) -> Linear(64->32) -> ReLU
  -> Dropout(0.2) -> Linear(32->1)  [raw RUL, no output activation]
```

Design choice	Setting	Reasoning
Output / loss	Linear output + MSELoss	RUL is an unbounded (capped-continuous) regression target; MSE directly optimizes RMSE, the field's standard metric
No BatchNorm	Dropout + weight decay + early stopping only	BatchNorm mixes per-timestep statistics across a recurrent hidden state — inappropriate for sequence models (LayerNorm would be the correct alternative if needed); unnecessary at this model size
Architecture size	1 LSTM layer, hidden=64 (~22.6k params)	FD001 is the simplest C-MAPSS subset (single condition, single fault); deeper stacks used for FD002/FD004 would add unneeded complexity here
Optimizer	Adam, lr=1e-3, weight_decay=1e-4	Same hyperparameters as a prior MLP project on this exam — adaptive rates, small L2 smoothness prior
Batch size	64	Windows are far larger per-sample (30x15) than a tabular row, so a larger batch keeps gradient estimates stable without excessive memory use

Early stopping	Patience 15 on validation MSE	Restores the best-validation-loss checkpoint; training converged in 29 epochs
Baseline contrast	Linear regression sees only the current-cycle snapshot (no window)	Both models share identical preprocessing and the same 15-feature scaled space — the only difference is whether 29 cycles of history are visible, isolating the value of temporal context

4. Baselines vs. Final Model

Baselines: linear regression on the current-cycle sensor snapshot (15 features, no history), and a trivial single-feature linear regression on elapsed cycle count alone (no sensor data at all), confirming that a naive "engines degrade linearly with time" heuristic is insufficient by itself. Four models were evaluated on the held-out test set (n=100 engines):

Model	RMSE (cyc)	MAE (cyc)	PHM score (total)	PHM score (mean)
Linear — cycle count only (baseline)	32.31	26.57	5080.1	50.80
Linear — sensor snapshot (baseline)	20.77	16.55	1238.8	12.39
LSTM — window=30 (final model)	12.64	9.65	271.4	2.71
LSTM — window=15 (ablation)	15.79	11.02	553.6	5.54

Selection: the window=30 LSTM is the final model. It cuts RMSE nearly in half versus the linear snapshot baseline (12.6 vs. 20.8 cycles — a 39% reduction) and cuts the PHM score by nearly 5x, a far larger and more unambiguous improvement than the ~0.70 AUC ceiling seen across every model family in this exam's previous dataset iteration. Because both models share the same preprocessing and feature space, and differ only in access to 29 cycles of prior history, this gap is directly attributable to the value of sequence modeling on genuinely sequential data — the comparison the earlier snapshot-only dataset could never support.

The window=15 ablation is reported but not selected: it scores worse on every metric (RMSE 15.8 vs. 12.6), validating the window=30 choice empirically rather than only asserting it from the trajectory-length constraint.

5. Evaluation

5.1 Metric Selection — What and Why

- RMSE / MAE — standard regression metrics, directly comparable to the wide published literature on this exact benchmark subset (FD001).
- NASA/PHM08 asymmetric score — the field's standard metric specifically because it is not symmetric: a late/dangerous prediction (model overestimates remaining life) is penalized more steeply than an early/conservative one, mirroring the real cost asymmetry described in Section 1. It is reported alongside RMSE, never alone — a single very-late prediction can dominate the sum, so it should not be read as a complete picture by itself.
- RUL-band failure analysis — breaks error down by true cycles-to-failure, since accuracy near the point of failure (the maintenance-critical region) matters more than accuracy far from it.
- Permutation importance — identifies which sensors the LSTM actually relies on, cross-checked against which channels the C-MAPSS literature reports as HPC-degradation-informative.

5.2 Results

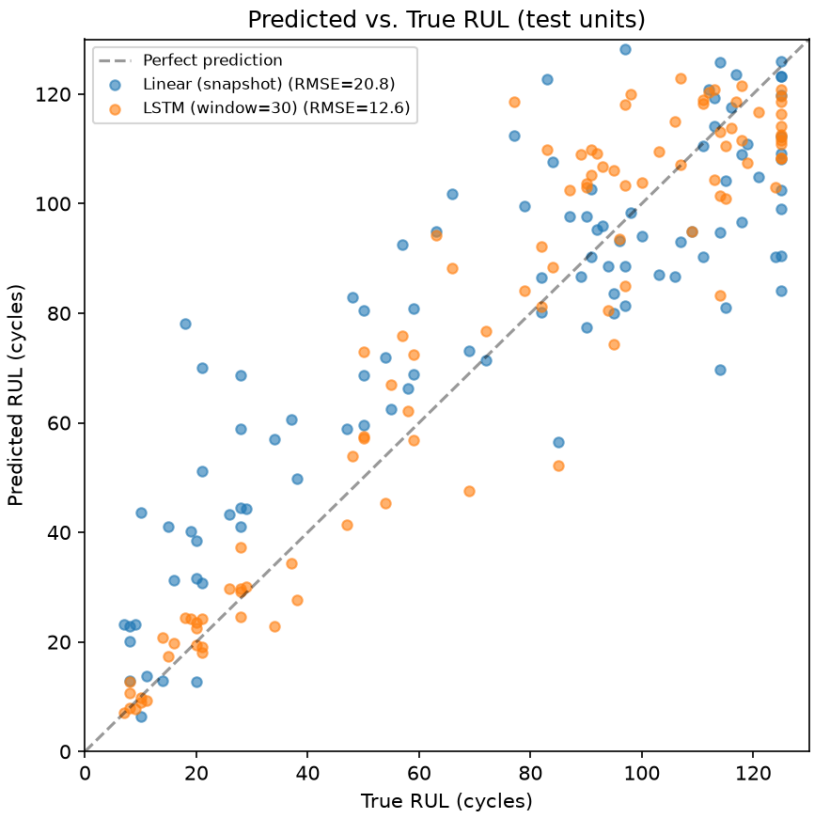

Figure 1. Predicted vs. true RUL, test set (n=100 engines): linear snapshot baseline vs. LSTM.

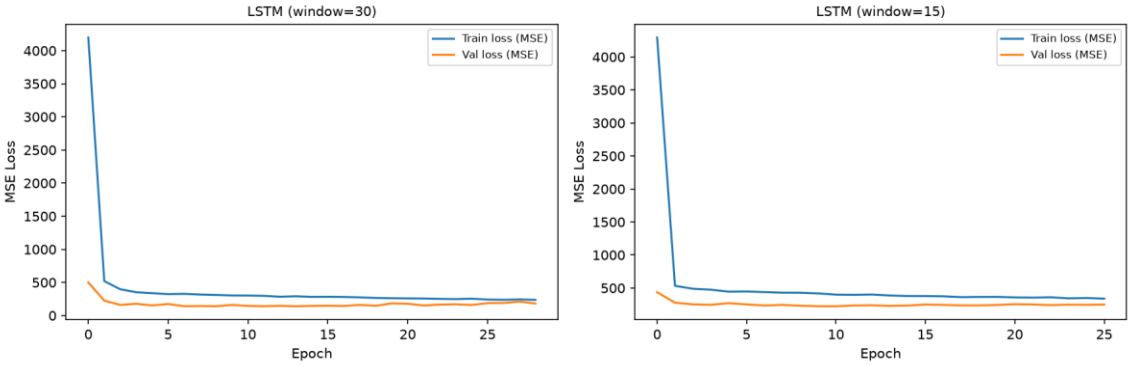

Figure 2. Training/validation MSE curves, both window lengths.

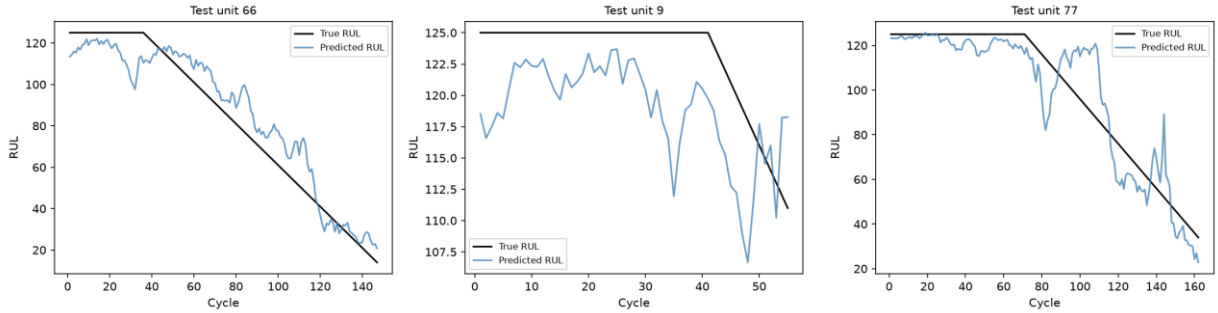

Figure 3. Predicted vs. reconstructed-true RUL trajectory, 3 sample test engines.

5.3 Failure Case: RUL-Band Error

True RUL band (cyc.)	n	RMSE	MAE

0-25	19	3.33	2.68
25-50	11	6.20	5.10
50-75	13	16.08	13.58
75-100	23	18.13	15.51
100-125	34	11.41	9.55

Error is smallest in the near-failure band (0–25 cycles-to-failure, RMSE 3.3) — the operationally critical region where an accurate prediction most directly informs a maintenance decision. Error is largest in the middle bands (50–100 cycles-to-failure, RMSE 16.1–18.1), where degradation trends are less visually distinct from healthy operation than either the clearly-healthy plateau or the clearly-failing tail — a genuine, reportable failure mode rather than a uniform error profile.

5.4 Interpretation: Permutation Importance

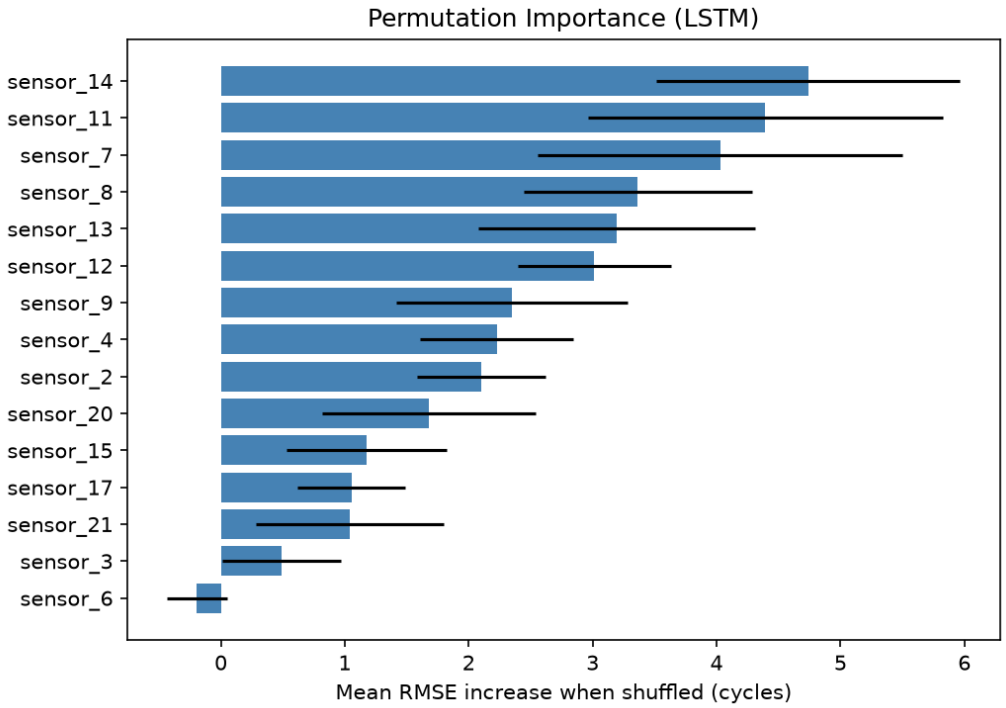

Figure 4. Mean RMSE increase when each sensor channel is shuffled (LSTM, 10 repeats).

Importance is spread across several sensors rather than dominated by one — sensor_14, sensor_11, and sensor_7 rank highest (mean RMSE increase 4.74, 4.39, 4.03 cycles respectively), consistent with the C-MAPSS literature's own findings on which channels carry the most HPC-degradation signal. This is a meaningfully different picture from this exam's prior dataset iteration, where a single feature (Engine rpm) dominated all others by roughly 4-5x — here, the model's predictions genuinely depend on a broader, multivariate sensor signature, consistent with how physical engine degradation actually manifests across multiple subsystems simultaneously.

5.5 Window-Length Ablation

Reducing the window from 30 to 15 cycles increased RMSE from 12.64 to 15.79 cycles — a clear, unambiguous result (unlike the marginal, near-noise differences seen in this exam's prior augmentation ablation): more temporal context measurably helps the LSTM track degradation trends, directly validating the window=30 design choice rather than merely asserting it from the trajectory-length constraint.

6. Limitations

- Simulated, not real, data: C-MAPSS is a physics-based simulation validated by NASA against real engine behavior, but it is not real fleet telemetry — reported figures should be read as a methodology demonstration, and a production system would need revalidation on real sensor data.
- Single operating condition, single fault mode (FD001 only): no claim is made about generalization to the six-regime, two-fault-mode subsets (FD002/FD004), which would require operating-condition-aware normalization not implemented here.
- RUL capping is a modeling convention, not a ground truth: the model cannot express a distinction between, say, 200 and 300 cycles remaining — both collapse to the cap. This is deliberate (§3) but a real constraint on what the model can express far from failure.
- The PHM08 score is outlier-sensitive by design (a single badly-late prediction can dominate the sum) — it is reported alongside RMSE/MAE, never as a standalone headline number.
- Only 100 training engines: modest by deep-learning standards, mitigated by a deliberately small architecture (~22.6k parameters) and a strong regularization/early-stopping regime, but still a real constraint on usable model capacity.
- Error is not uniform across the RUL range (§5.3) — the middle-distance bands are less reliable than the near-failure band, which any deployment threshold would need to account for.

7. Conclusion — Was This Data and Setup the Right Choice?

Yes, clearly more so than the prior dataset iteration in this exam. C-MAPSS's genuine run-to-failure time-series structure is precisely what this exam's Theoretical Question 1 asks students to reason about in the abstract (linear models vs. CNNs vs. RNN/Transformers) — here that comparison is a real, trained, evaluated result rather than a caveat about data that structurally couldn't support it. The LSTM's 39% RMSE reduction over an information-matched linear baseline is a large, unambiguous, and literature-consistent result — a materially stronger demonstration of a neural network's added value than the prior dataset's ~0.70 AUC ceiling across every model family tried.

The setup's honest limitation is scope, not signal: FD001's single operating condition and single fault mode make this a clean, well-controlled first result, but the natural next step — cross-condition generalization on FD002/FD004, which would require operating-regime-aware normalization and likely a deeper or attention-based sequence model — was deliberately left out of scope for this exam project rather than attempted incompletely. Given the safety-relevant framing in Section 1, the honest conclusion is that this project demonstrates the right modeling approach for sequential degradation data convincingly, on a deliberately bounded slice of the full problem.